

Declan Sheehan

Dr. Anderson

COSC320-001

13, November 2019

Lab 9 README

Approach:

My approach to this lab is generalized by the following:

- Followed the pseudocode
- Repurposed Lab 8 functions
- Kept in mind the attributes of a red black tree
- Used newer commands to find deallocation issues, such as:
 - Valgrind --leak-check=full -v ./lab9.exe

Theoretic Time Complexity:

Averages:

	RBT	BST
Insert:	$O(\log n)$	$O(\log n)$
Delete:	$O(\log n)$	$O(\log n)$
Search:	$O(\log n)$	$O(\log n)$

For Binary Search Tree, worst case is $O(n)$ for above functions.

For Red Black Tree, worst case is $O(\log n)$ for above functions.

Red Black Tree VS Binary Search Tree Operations:

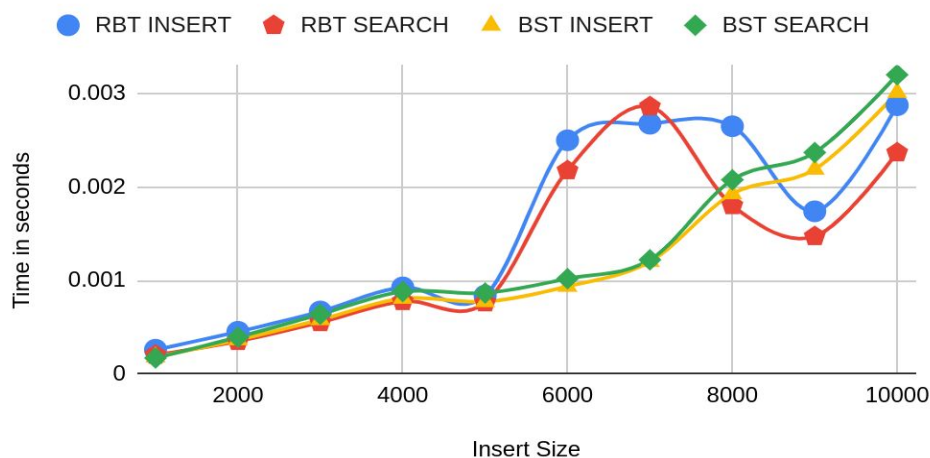
Ascending Operations RBT Vs. BST.



Ascending Insertion RBT Vs. BST (Subgraph)



Random (Order) Operations RBT Vs. BST



Judging from the above graphs collected from the tests, RBT will eventually out-perform a regular BST in insertion and search operations. If we are to believe the worst case for BST is $O(n)$, we can see that RBT takes a fair bit less, so $O(\log n)$ for RBT makes total sense for its time complexity.

Data Tables:

Insert Size	RBT INSERT	RBT SEARCH	BST INSERT	BST SEARCH
1000	0.000384389	0.000231568	0.00549893	0.0084586
2000	0.000070397	0.000495383	0.0241955	0.0384736
3000	0.000108592	0.000783538	0.0576503	0.100328
4000	0.000144699	0.00106156	0.141716	0.157475
5000	0.00186244	0.00136826	0.151194	0.359782
6000	0.00205467	0.00152749	0.303314	0.385109
7000	0.0023909	0.00182485	0.326561	0.5162
8000	0.00274822	0.00193544	0.440721	0.619663
9000	0.00299866	0.00223464	0.566646	0.787337
10000	0.00317657	0.00232102	0.706213	1.06223

Insert Size	RBT INSERT	RBT SEARCH	BST INSERT	BST SEARCH
1000	0.000258462	0.000202854	0.000181126	0.000170359
2000	0.000450867	0.00034974	0.000364075	0.000393397
3000	0.000668398	0.000551248	0.000579088	0.000638229
4000	0.00092589	0.000776875	0.000808295	0.000881815
5000	0.000838566	0.000763377	0.000772824	0.000865728
6000	0.00250037	0.00217726	0.000936566	0.00101951
7000	0.00267785	0.00286211	0.00120203	0.00122144
8000	0.0026508	0.00180341	0.00192724	0.00207773
9000	0.00174013	0.00147215	0.00219	0.00236958
10000	0.00287576	0.00236873	0.0030122	0.00320115

Code Improvements:

Even though the delete node works, It stills seg-faults when testing the delete function for > 1000 elements. It is either that or it leaks data; I can't remember. Fixing that would be "good".